

Judiciously Spreading Approximation Among Arithmetic Components with Top-Down Inexact Hardware Design

Giovanni Ansaloni(✉), Ilaria Scarabottolo, and Laura Pozzi

Università della Svizzera Italiana, Lugano, Switzerland
{giovanni.ansaloni,ilaria.scarabottolo,laura.pozzi}@usi.ch

Abstract. Approximate logic synthesis is emerging as a promising avenue towards the development of efficient and high performance digital designs. Indeed, effective methodologies for the inexact simplification of arithmetic circuits have been introduced in recent years. Nonetheless, strategies enabling the integration of multiple approximate components to realise complex approximate hardware modules, able to maximise gains while controlling ensuing Quality-of-Service degradations, are still in their infancy. Against this backdrop, we herein describe a methodology to automatically distribute the error leeway assigned to a hardware design among its constituent operators. Our strategy is able to identify high-quality trade-offs among resource requirements, performance and exactness in digital implementations, across applications belonging to different domains, and without restrictions on the type and bit-width of their approximable arithmetic components.

Keywords: Inexact computing · Approximate logic synthesis · Electronic design automation

1 Introduction

Established methodologies for hardware development explore trade-offs between performance (latency, throughput) and cost (area, energy) of a target design. Approximate Logic Synthesis challenges this paradigm by also considering a further dimension: that of the desired degree of *exactness*. ALS frameworks simplify those parts of a design that have a small impact on the performed computation, deriving inexact, but more efficient, implementations. ALS can be effectively applied in domains ranging from signal processing to machine learning [9], where careful approximation-induced perturbations lead to graceful Quality-of-Service (QoS) degradations.

This work has been partially supported ML-edge (grant no. 200020_182009 - 156397) project funded by the Swiss NSF and the MyPreHealth (grant no. 16073) project funded by Hasler Stiftung.

© Springer Nature Switzerland AG 2020
F. Rincón et al. (Eds.): ARC 2020, LNCS 12083, pp. 14–29, 2020.
https://doi.org/10.1007/978-3-030-44534-8_2

ALS has been the focus of an increasing number of research efforts in recent years [14,15,18]. Nonetheless, most works in this field target the simplification of single arithmetic circuits. To fully realise the potential of ALS, it is instead critical to establish strategies that integrate multiple, individually-tailored, inexact elements to realise complex (inexact) hardware modules. Against this backdrop, we here introduce a methodology, named TDApprox, that automatically distributes the error tolerable at the output of a hardware design among its constituent operators, hence driving their inexact synthesis.

Other recent works addressing this challenge assume, as a required input to their frameworks, the availability of a library of approximated components, which are then selected and integrated [1,2,6,10,16]. As illustrated on the left side of Fig. 1, such bottom-up stance (in which the characterisation of operators is performed a-priori, independently from the design implementation) poses a limit on flexibility, because (a) for a given operator, library instances could expose approximation levels which are not compatible with the ones required by a target design, or (b) the accelerator to be simplified may require input/output bitwidths not available in characterised components. Finally (c), and most importantly, an operator could even be entirely missing from the employed library.

To avoid these pitfalls, TDApprox instead embodies a top-down strategy (hence its name). As described on the right side Fig. 1, in our approach the ALS of operators is only performed *after* assessing their approximability, i.e.: the impact of operators-level approximations on the overall design Quality-of-service

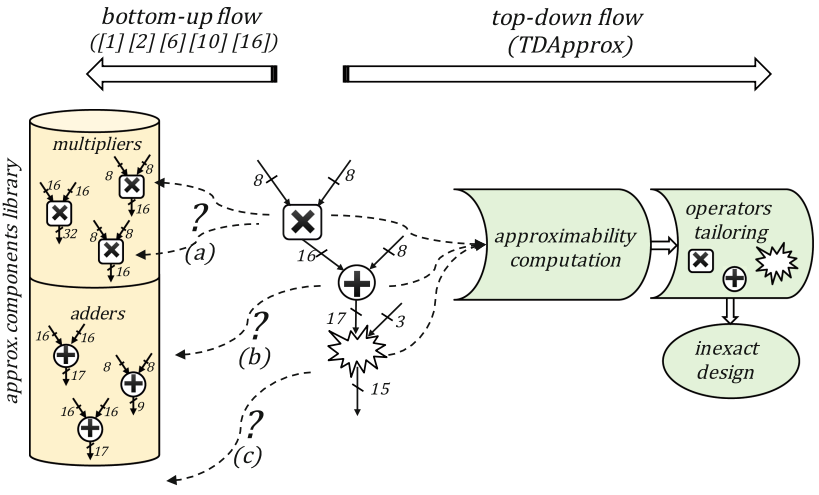


Fig. 1. Comparison between a bottom-down approach (left), selecting approximated components from a library, and the top-down stance embodied by TDApprox (right), in which the evaluation of the approximability of arithmetic operators is precursory to their synthesis.

(QoS). TDApprox is therefore not limited to a pre-determined set of operators and I/O bitwidth, and can consider arbitrary degrees of approximation.

TDApprox interfaces to state-of-the-art methods for the ALS of single arithmetic circuits, automatically deriving synthesisable hardware netlists of complex inexact accelerators, from C code and an error bound on the design output.

In summary, our contribution is three-fold:

- We introduce a top-down methodology for distributing a user-defined approximation leeway among the arithmetic operators comprising a hardware design.
- We detail how TDApprox is integrated in an ALS framework for the synthesis of inexact designs, starting from a high-level (C) description of their functionality.
- We showcase the effectiveness of TDApprox on applications from diverse error-resilient fields, considering both combinatorial and sequential implementations.

The paper proceeds as follows: Sect. 2 further places our contribution in the context of related works. Section 3 details the TDApprox methodology, while Sect. 4 assesses its performance. Section 5 concludes the paper.

2 State of the Art

Various automated strategies for the inexact synthesis of generic arithmetic circuits have been proposed in literature. Notably, Venkataramani et al. proposed in [19] to approximate logic cones based on don't care analysis, while in [18] introduced an algorithm to identify and merge sub-circuits performing similar functions. Schlachter et al. [15] instead adopt a significance and an activity metric to guide logic simplification, Ranjan et al. [13] is based on model checking, and Nepal et al. [12] enumerate inexact transformations modelled as abstract syntax trees. Finally, Scarabottolo et al. [14] base their approach on a branch-and-bound algorithm, identifying the largest sub-circuit that can be simplified from a gate-level netlist under an error constraint.

Occasionally, the authors of these works provide examples of how inexact operators realised using their methodologies can be integrated: [15] and [13] study the performance of a DCT accelerator implemented using their approximated components, while [12] consider a perceptron classifier and a block matcher. Nonetheless, in all those cases the accelerators implementations are had-hoc, and no strategy is suggested to systematically compose inexact building blocks.

The few works that aim at addressing the ensuing integration problem do so, as opposed to our methodology, from a bottom-up perspective: Du et al. [2] rely on a pre-characterised set of seven inexact multipliers to drive their heuristic, Li et al. [6] consider five inexact adders and multipliers, which are selected by either a knapsack algorithm or an ILP formulation. Sengupta et al. [16] and Chan et al. [1] analytically derive the errors and resource requirements of simplified adders and multipliers. Mrazek et al. [10] adopts a machine learning approach to

compose the inexact operators included in the EvoApprox8b library [11], which provides three adders and eight multipliers at varying approximation levels.

3 Methodology

Input to TDApprox is a directed and acyclic Data Flow Graph (DFG) $G(N, E)$, representing the exact functionality to be approximated. Nodes n_i represents arithmetic operations and edges $e(n_i, n_j)$ data dependencies among them. Additionally, the edge set E_{in} accounts for the DFG inputs; e_{out} for its output[1]. Furthermore, input edges are annotated with the lower and upper bounds of their admissible values: $x_i = [l_i, u_i]$. As and example, $x_i = [0, 255]$ for an 8-bit unsigned value.

The DFG is traversed to analyse the effect, as seen by e_{out}, of perturbations induced by each node. As an illustrative example, consider the simple case in Fig. 2a, where the output of an approximate adder is connected to one input of an exact multiplier. If the adder output differs from that of an exact addition by δ_A, the error at the DFG output is, in the worst case, $\delta_A \times \max(|x_3|)$, hence magnifying the approximation degradation if $\max(|x_3|) > 1$. Note that $\max(|x_3|)$ can be either $-l_3$ or u_3 depending on the bounds on its admissible values.

As detailed by Misailovic et al. [8], if a node $n_i \in N$ performing a generic arithmetic operation f with input edges $e_{1,i}(n_{j1}, n_i)$ and $e_{2,i}(n_{j2}, n_i)$ has inexact predecessors, and it is itself inexact, the error at its output is bounded as follows:

$$|f_i(x1, x2) - \hat{f}_i(\hat{x1}, \hat{x2})| \leq k_{1,i}|err(e_{1,i})| + k_2|err(e_{2,i})| + |\delta_i| \tag{1}$$

where $f(x1, x2)$ and $\hat{f}(\hat{x1}, \hat{x2})$ are the node output values in the exact and the inexact case (respectively), $err(e_1)$ and $err(e_2)$ represent the impact of approximating the predecessors of n_i, and δ_i is the approximation introduced by n_i itself. Moreover, the propagation coefficients k_1 and k_2 indicate how the inexactness at the input of the node are magnified or dampened by the performed function f, and are defined as:

$$k_1 = \max_{x1,x2} \left|\frac{\partial f(x1, x2)}{\partial x1}\right| \quad k_2 = \max_{x1,x2} \left|\frac{\partial f(x1, x2)}{\partial x2}\right| \tag{2}$$

The solution of Eq. 2 for common arithmetic operations is presented in Table 1.

We apply such formulation in a different context with respect to Misailovic et al. [8] [2] i.e., the approximate synthesis of hardware accelerators. In our scenario, the error caused by an approximate node n_i (with approximation bound δ_i) must be evaluated at the DFG output e_{out}. Therefore, we define the induced

[1] For simplicity, we focus on single-output DFGs. Multiple-outputs cases can nonetheless be addressed by considering each output in isolation, and then selecting the most stringent approximation constraint for each operation.

[2] The authors of [8] aim to bind approximable operations (with known approximability) to inexact functional units.

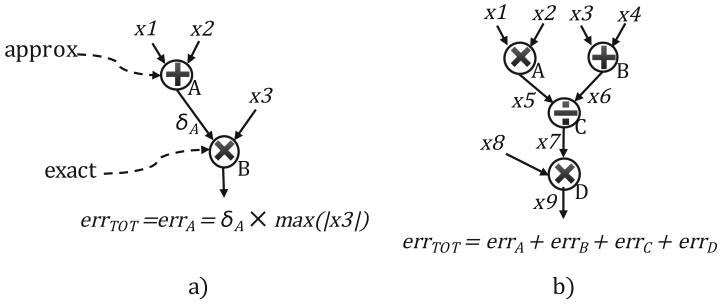


Fig. 2. Right: DFG with one approximated node. Left: DFG having all nodes implemented as approximate operators.

error bound of an operation as err_i, which is the product of the propagation coefficients along the path connecting n_i to e_{out}, if only one such path exist:

$$err_i = \delta_i \times \prod_{n_j \in succ(n_i)} k(n_j) \tag{3}$$

where $succ(n_i)$ are the successor nodes of n_i in G, considered recursively.

When instead several paths between n_i and e_{out} are present, err_i is bounded by the sum of the errors induced along the different paths:

$$err_i = \delta_i \times \sum_{p \in paths} \left(\prod_{n_j \in succ(n_i)} k(n_j) \right) \tag{4}$$

Finally, when *all* nodes in G are inexact to a degree, the deviation from an exact formulation is the super-position of the errors due to the approximation of each operation:

$$err_{TOT} = \sum_{n_i \in G} err_i = \sum_{n_i \in G} [\delta_i \times \sum_{p \in paths} \left(\prod_{n_j \in succ(n_i)} k(n_j) \right)] \tag{5}$$

err_{TOT} is the bound to the overall quality degradation at the DFG level, set according to the application constraint. This leeway can then be distributed among the err_i coefficients.

Table 1. Propagation coefficients for common operations.

Operation	k_1	k_2
Addition/subtraction (+/−)	1	1
Multiplication (×)	$\max(\lvert x_2 \rvert)$	$\max(\lvert x_1 \rvert)$
Division (÷)	$\max(1/\lvert x_1 \rvert)$	$\max(\lvert x_1/x_2^2 \rvert)$

If $err_1 = ... = err_N = err_{TOT}/card(N)$, the error induced by each approximate operation equally affects the output QoS degradation. While this may seem a sensible choice, operators may have different resource requirements e.g.: a high bit-width multiplier being much more area- and power-hungry than a small-bitwidth adder. Consequently, TDApprox assigns more error slack to the operators having a higher Energy-Area-Delay Product (EDAP) in their exact implementation:

$$err_i = err_{TOT} \times \frac{EDAP(i)}{\sum_{n \in N} EDAP(n)} \tag{6}$$

Fixing the values of the err_i coefficients allows to solve the Eq. 4 for $\Delta = \{\delta_1, ..., \delta_N\}$, i.e., to compute the error bound for each operation in G. As shown below, such equations are not independent from each other, because the propagation factors k pertaining to a DFG node depend on the admissible intervals of its inputs (as defined in Table 1), themselves influenced by the perturbation introduced by the node predecessors.

Example. Consider the DFG in Fig. 2b, composed of 4 nodes (n_A, n_B, n_C, n_D), and whose inputs are bounded in the intervals $x1, x2, x3, x5, x4, x8$. The first input of each operation is depicted as the left edge, the second input as the right edge. For each node, Eq. 4 is as follows:

$$\begin{cases} err_A = \delta_A \times k_{1,C} \times k_{2,D} \\ err_B = \delta_B \times k_{2,C} \times k_{2,D} \\ err_C = \delta_C \times k_{2,D} \\ err_D = \delta_D \end{cases} \tag{7}$$

Using the propagation rules in Table 1 for all k values:

$$\begin{cases} err_A = \delta_A \times \max(|1/x6|) \times \max|x8| \\ err_B = \delta_B \times \max(|x5/x6^2|) \times \max|x8| \\ err_C = \delta_C \times \max|x8| \\ err_D = \delta_D \end{cases} \tag{8}$$

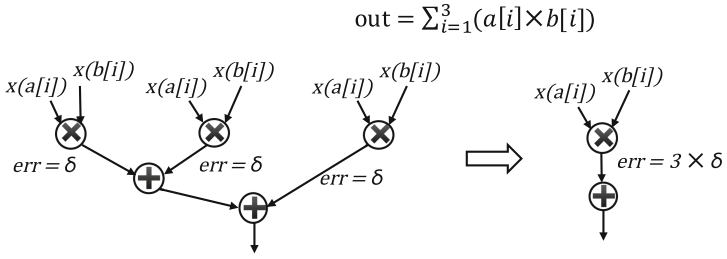


Fig. 3. Coalescing iterations in an accumulation loop.

The intermediate values x5 and x6 can be expressed as a function of DFG inputs and errors induced by the approximation of operators proceeding them, back-traversing the DFG:

$$x5 = (x1 \times x2) + \delta_A \qquad x6 = (x3 + x4) + \delta_B \tag{9}$$

Substituting these expressions in Eq. 8 gives the final form of a system of equations, with err_i parameters and δ_i unknowns:

$$\begin{cases} err_A = \delta_A \times \max\left(|1/(x3 + x4 + \delta_B)|\right) \times \max|x8| \\ err_B = \delta_B \times \max\left(|(x1 * x2 + \delta_A)/(x3 + x4 + \delta_B)^2|\right) \times \max|x8| \\ err_C = \delta_C \times \max|x8| \\ err_D = \delta_D \end{cases} \tag{10}$$

Solving the unknowns provides the approximation leeway for each DFG node.

Applicability. The TDApprox approach can be applied to all cases where a DFG, representing the computation in an approximate hardware datapath, can be obtained. Such representation is derived straightforwardly from a software description when no control statements are present, or when these are flattened (e.g.: fully unrolling every loop) [5,17]. An equivalent DFG can also be constructed when statically-defined (e.g.: `for`) loops are employed to update an accumulator register, as exemplified in Fig. 3. This case, common in approximable applications (as the one investigated in Sect. 4.3), is addressed by only considering the operations in a single iteration of the loop bodies, with the maximum input/output bitwidths required considering all iterations. Since the accumulator variables also accumulate the error due to the inexactness of their predecessors, the *err* value at their inputs must then be multiplied by the number of loop iterations.

4 Experimental Evaluation

4.1 Experimental Setup

We interfaced the error distribution algorithm described in Sect. 3 with a front-end deriving the DFG of the targeted applications, and a back-end performing the approximation of operators according to the identified leeways. A block scheme of such experimental environment is illustrated in Fig. 4.

The framework processes the source code description of an accelerator functionality, expressed in C and containing annotations that specify the maximum and minimum admissible value of the accelerator inputs. The corresponding DFG, as well as the DFG nodes types and their input/output bitwidths, are extracted employing dco-scorpio [17]. We leveraged this information to automatically generate a behavioural verilog descriptions of the DFG nodes, which are then synthesised to obtain (a) the operators EDAP required by the TDApprox algorithm and (b) the operators gate level netlists to be simplified.

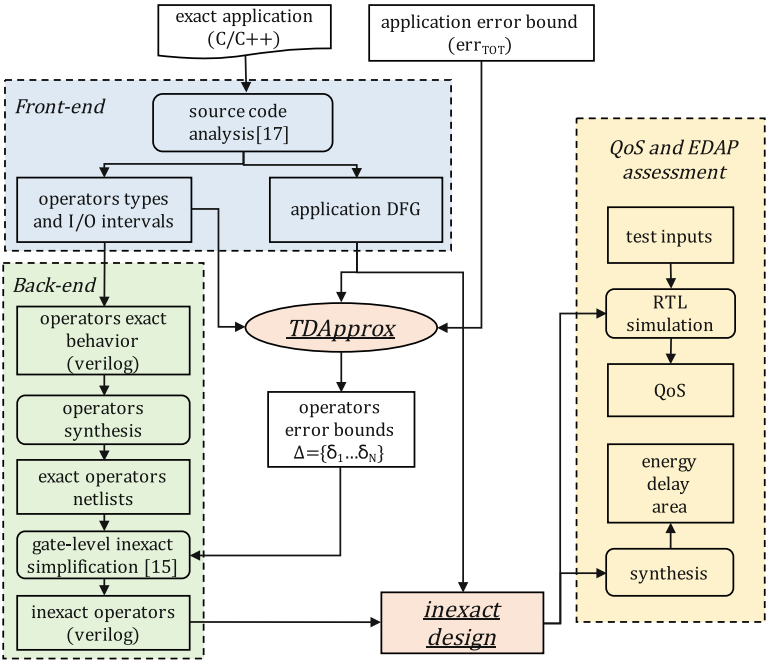


Fig. 4. Experimental framework. The front-end provides a DFG representation of the source code, while the back-end individually tailors the operators according to the Δ bounds retrieved by TDApprox, and perform the synthesis of the resulting inexact design.

Output of TDApprox are error bounds for each operation. They are employed in the framework back-end to individually tailor each corresponding netlist by removing their least critical gates. For this step, we re-implemented the Gate Level Pruning (GLP) strategy illustrated in [15], but any equivalent methodology (e.g., [14,18]) could be similarly used. Output of our GLP implementation are inexact hardware arithmetic operators, described (again, in verilog), as a series of statements expressing bit-wise operations.

Finally, these are integrated and synthesised to evaluate their area, critical path delay, and energy. Moreover, they are simulated to gauge their Quality-of-Service. We employed Synopsys Design Compiler (considering a 40 nm technology library) as a synthesis tool, and Menthor Graphics Moldesim for simulations. The generation of test benches is not automated at this time; nonetheless, since simplification does not affect the operators interface (but only their internal structure), the same test bench can be employed for both exact and inexact implementations, or to compare different inexact variants of the same design.

Table 2. Benchmarks characteristics.

B.mark	#ops	#mults	mult bit-width	#adds	adds bit-width	#divs	divs bit-width
`Sobel`	45	0	-	26	9, 10, 11, 12	9	8
`mult-FIR`	50	24	9, 10, 13, 14, 16	12	10, 11, 13, 15, 16	0	–
`SVM`	9	3	16, 27, 30	4	14, 21, 37, 38	0	–

B.mark	Implementation	Timing constraint	Test inputs
`Sobel`	Combinatorial	2 nS	100×100 RGB image
`mult-FIR`	Combinatorial	1 nS	2×256 points signals: $\sin(at)$, $\sin(bt^2)$
`SVM`	Sequential	2 nS	ECG features

We evaluated TDApprox on three benchmark applications belonging to the domains of image/signal processing and classification, fields that typically exhibit a degree of resilience towards judicious exactness degradations. Further illustrating our framework flexibility, we considered both combinatorial implementations and sequential ones. Similarly to [11,16], we assessed the performance of our methodology with respect to a uniform baseline strategy that assigns the same error (δ_i) to all operators. More details describing the benchmarks are provided in Table 2; for brevity, only the output widths of operators are reported, even if in all cases input widths also vary widely, and are not symmetric. This characteristic, along with the presence of operations other than multiplications and additions, makes these applications ill-suited for bottom-up methods relying on pre-characterised inexact components. As an example, [11] also considers a Sobel accelerator, but the implementation described therein misses the RGB averaging step, because the required operator (divide-by-3) is not available in their employed library [10].

4.2 Combinatorial Designs

The `Sobel` benchmark averages the red, green and blue components of 3×3 neighbourhoods of pixels in an input image, and convolves them with a kernel matrix to obtain a blurred greyscale output:

$$out = (R_{3\times3} + G_{3\times3} + B_{3\times3})/3 * (\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}) \tag{11}$$

The `mult-FIR` benchmark instead operates on sliding windows of 12 elements, multiplying element-wise the samples of two input signals, which are then fed to a 12-taps low-pass FIR filter:

$$out = ((sig1_{12\times1} \odot sig2_{12\times1})/128) \cdot (\frac{1}{256}[-2,-2,3,18,42,61,61,42,18,3,-2,-2]) \tag{12}$$

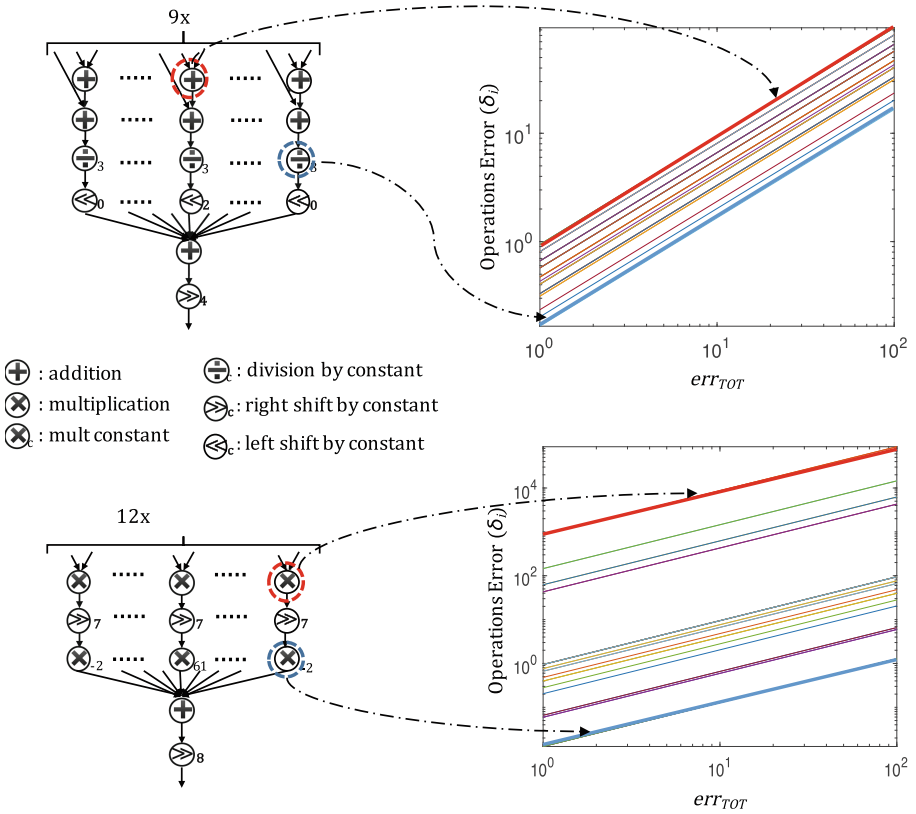


Fig. 5. Left: Schemes of the `Sobel` (top) and the `mult-FIR` (bottom) benchmarks. Right: Respective operations errors δ_i computed by TDApprox, varying the output error bound err_{TOT}.

In both cases, inputs are represented as 8-bit values. The structure of the resulting datapaths are depicted on the left side of Fig. 5. Moreover, the right side of the figure reports the inexactness bounds $\Delta = \{\delta_1, ..., \delta_N\}$ assigned by TDApprox to each arithmetic operator for varying datapaths error bounds err_{TOT}, obtained by solving Eqs. 4 and 6 for the two DFGs.

In the case of the `Sobel` benchmark (top), one order of magnitude separates the δ of the least and the most critical operations i.e., the ones assigned the highest and lowest leeway for approximation (highlighted in the figure). As expected, the operations averaging RGB components at the corners of the input neighbourhood, which are multiplied by 1 using the kernel matrix, are assigned a higher δ with respect to the ones averaging the central pixel, which is multiplied by 4, hence magnifying any perturbation.

A similar observation can be made for the `mult-FIR` case (bottom of Fig. 5): TDApprox assigns a higher δ to the multipliers operating on the extremes of the sliding window, which correspond to smaller FIR coefficients. Moreover, when

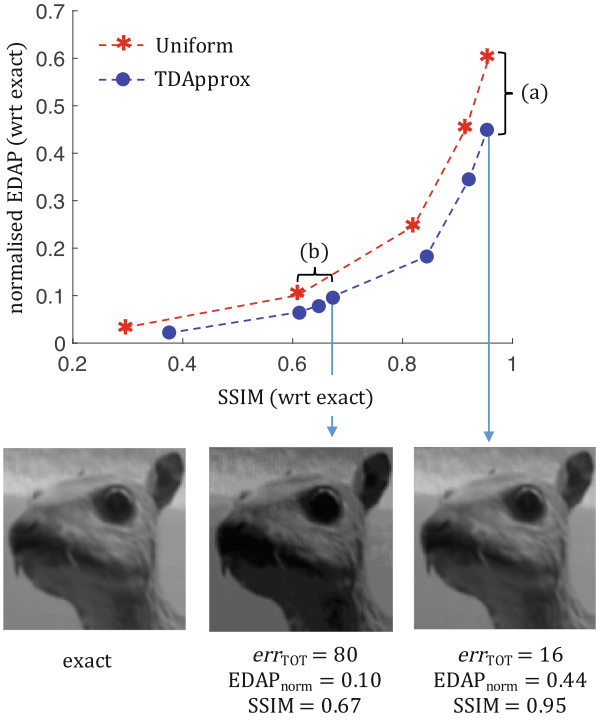


Fig. 6. EDAP-QoS trade-offs achieved by TDApprox and uniform error distributions when approximating the `Sobel` datapath.

examining the adder chain accumulating the final output, a smaller δ is assigned to the first stages (which require a smaller bit-width, hence fewer resources) with respect to the latter ones. In the case of `mult-FIR`, the most approximable operation is assigned 900X more slack than the least approximable one.

Such non-uniform error distribution results in high-quality resource-QoS trade-offs at the application level, showcased in Fig. 6 for the `Sobel` benchmark. Exact and approximated outputs (for two approximation levels) are depicted at the bottom of the figure, highlighting that a normalised Energy-Delay-Area Product of 0.1 (a 10X reduction) can be achieved with little impact on the perceived output quality. In its upper part, the figure reports the Pareto curve of the best performing inexact implementations retrieved using TDApprox, plotting their EDAP on the x axis and their QoS on the y axis. The latter is expressed as achieved Structural Similarity Index (SSIM) with respect to the exact output. The characteristics of the best performing implementations employing the same error bound in all operators are also presented. Compared with this baseline strategy, TDApprox achieves (a) 34% higher reduction in EDAP for an SSIM of 0.95, and (b) 10% higher SSIM for a normalised EDAP of 0.1.

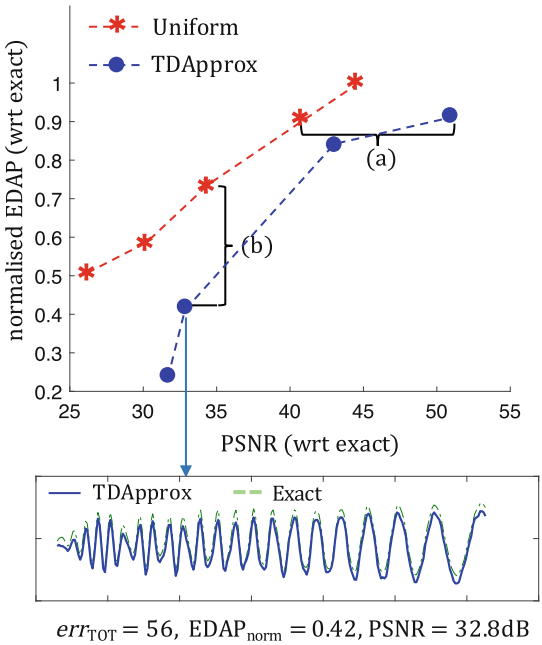


Fig. 7. EDAP-QoS trade-offs achieved by TDApprox and uniform error distributions when approximating the `mult-FIR` datapath.

The effectiveness of TDApprox is even more apparent for the `mult-FIR` benchmark (Fig. 7), because it presents higher variations in the approximability of operators. In this case, we considered the Peak Signal-to-Noise Ratio (PSNR) between exact and inexact outputs as a QoS metric. Indeed, when compared to a uniform distribution of error leeways among DFG operations, the bounds computed by TDApprox result in implementations having (a) a 41% increase in PSNR for the same EDAP, or alternatively (b) an increase in EDAP savings of 44% for a PSNR difference of only 3%.

4.3 Sequential Design

As an example of a sequential design, we targeted the inference calculation of a quadratic SVM machine. The SVM determines which features set, extracted from 2-min electrocardiogram (ECG) windows, were acquired during epileptic episodes. Features were derived according to the methodology described in [4]. The clinical data processed in this set of experiments, provided by CHUV university hospital (Lausanne, Switzerland), is divided in 24 folds, containing 10 seizure and 10 non-seizure windows. Cross-fold validation was performed, employing in each round one fold for test and the rest of the database for training, then averaging the results.

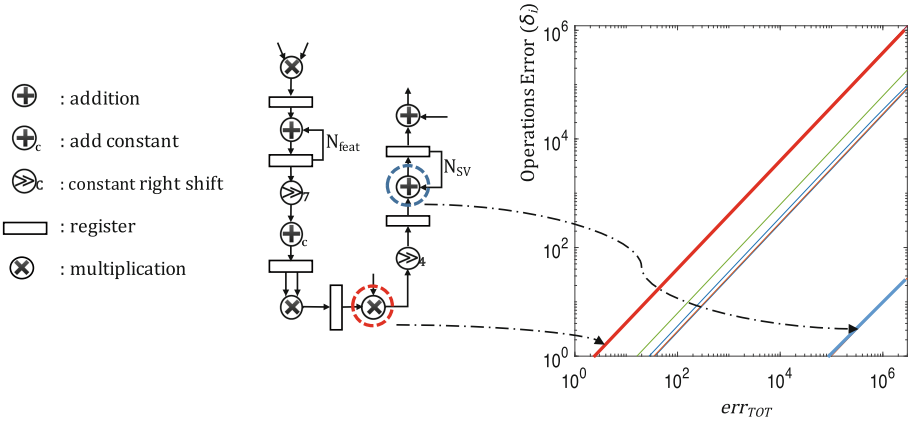


Fig. 8. Left: Block scheme of the `SVM` datapath. Right: Operations errors δ_i computed by TDApprox, varying the output error bound err_{TOT}.

The inference formula for quadratic SVMs is as follows:

$$Y = \sum_{i \in SVs} (\alpha_i y_i \ (\mathbf{x}_T \cdot \mathbf{x}_i + 1)^2) + b \tag{13}$$

where the sign of Y discriminates among classes.

As an exact baseline, we consider an aggressively tailored design employing reduced bitwidths in all datapath stages, with the aim to explore the *further* savings attainable by TDApprox, beyond the ones deriving from the truncation of Least Significant Bits (LSBs). In the baseline implementation, inputs are quantised to 8 bits, while 7 LSBs are dropped after the dot-product stage and 4 LSBs before the summation stage. As reported by the authors of [7] such setting, with respect to a floating-point equivalent, resulted in a negligible ($< 1\%$) impact on QoS, defined for the `SVM` benchmark as the geometric mean of the achieved Sensitivity (Se) and Specificity (Sp) [3]: $GM = (Se \times Sp)^{1/2}$.

The datapath, whose block scheme is provided in Fig. 8, presents a first feedback loop to accumulate the dot-product between the test feature vector $\mathbf{x}_T$ and a support vector $\mathbf{x}_i$, and a second one to accumulate the terms of the summation in Eq. 13. The figure also plots the Δ thresholds of its operations inexactness in dependence of the error bound at the datapath output Y. Again, orders of magnitude separate the most and the least approximable operation.

In our implementation, 15% of the hardware real estate is devoted to the control logic, which is not amenable to approximation. Even when this overhead is accounted for, approximate logic synthesis driven by TDApprox leads to tangible gains. As shown if Fig. 9, a 21% reduction in EDAP is attainable for a negligible GM degradation of 0.5% (a). The quality of the obtained approximated implementations dwarves the one attainable with a homogeneous distribution of errors: by smartly assigning error bounds, TDApprox achieves both a better

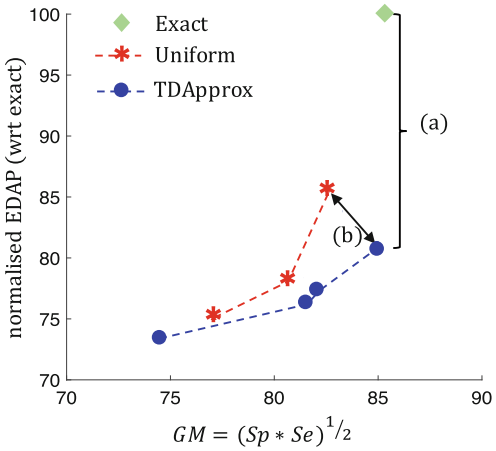


Fig. 9. EDAP and QoS of TDApprox-driven implementations with respect to an exact one, and alternatives assigning the same error to all operators.

Table 3. Performance of inexact implementations, using TDApprox vs. a uniform distribution of error bounds.

TDApprox	err_{TOT}	$EDAP_{norm}$	Sp	Se	GM
	1000	81.02	81.99	88.04	84.96
	2000	83.78	80.43	85.87	83.11
	5000	76.42	73.91	89.86	81.50
	10000	73.51	60.25	92.03	74.46
Uniform	δ_i	$EDAP_{norm}$	Sp	Se	GM
	100	85.6	83.23	81.88	82.55
	200	78.24	82.30	78.99	80.62
	300	75.13	74.53	79.71	77.08

classification performance (+2.5% GM between the points marked as (b) in the figure) and lower resource requirements (−6% EDAP). Table 3 further details the experimental outcomes, reporting the achieved performance for varying err_{TOT} bounds (in the case of TDApprox) and for different assignment of δ_i inexactness level (for the baseline uniform methodology).

5 Conclusion

We presented TDApprox, a top-down methodology for the design of complex inexact hardware modules. By distributing approximation leeways to arithmetic operators based solely on a description of the intended functionality and on a threshold to the overall exactness, TDApprox can effectively drive their

automated synthesis. Hence, our methodology is not bound by a pre-determined library of approximable operators nor an a-priori choice of approximation levels.

Such approach seamlessly enables to target diverse application fields and QoS metrics, identifying highly effective resource/quality trade-offs.

References

1. Chan, W.T.J., Kahng, A.B., Kang, S., Kumar, R., Sartori, J.: Statistical analysis and modeling for error composition in approximate computation circuits. In: Proceedings of the International Conference on Computer Design, pp. 47–53, October 2013
2. Du, Z., Palem, K., Lingamneni, A., Temam, O., Chen, Y., Wu, C.: Leveraging the error resilience of machine-learning applications for designing highly energy efficient accelerators. In: Proceedings of the Asia and South Pacific Design Automation Conference, pp. 201–206, January 2014
3. Fleming, P.J., Wallace, J.J.: How not to lie with statistics: the correct way to summarize benchmark results. Commun. ACM **29**(3), 218–221 (1986)
4. Forooghifar, F., et al.: A self-aware epilepsy monitoring system for real-time epileptic seizure detection. Mob. Netw. Appl. 1–14 (2019)
5. Lattner, C., Adve, V.: LLVM: a compilation framework for lifelong program analysis & transformation. In: Proceedings of the 2nd International Symposium on Code Generation and Optimization, pp. 75–88, March 2004
6. Li, C., Luo, W., Sapatnekar, S.S., Hu, J.: Joint precision optimization and high level synthesis for approximate computing. In: Proceedings of the 52nd Design Automation Conference, pp. 1–6, June 2015
7. Ferretti, L., et al.: Tailoring SVM inference for resource-efficient ECG-based epilepsy monitors. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition, pp. 1–4, March 2019
8. Misailovic, S., Carbin, M., Achour, S., Qi, Z., Rinard, M.C.: Chisel: reliability-and accuracy-aware optimization of approximate computational kernels. In: SIGPLAN Notices, vol. 49, pp. 309–328. ACM, October 2014
9. Mittal, S.: A survey of techniques for approximate computing. ACM Comput. Surv. (CSUR) **48**(4), 62:1–62:33 (2016)
10. Mrazek, V., Hanif, M.A., Vasicek, Z., Sekanina, L., Shafique, M.: autoAx: an automatic design space exploration and circuit building methodology utilizing libraries of approximate components. In: Proceedings of the 56th Design Automation Conference, p. 123, June 2019
11. Mrazek, V., Hrbacek, R., Vasicek, Z., Sekanina, L.: EvoApproxSb: library of approximate adders and multipliers for circuit design and benchmarking of approximation methods. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition, pp. 258–261, March 2017
12. Nepal, K., Li, Y., Bahar, R., Reda, S.: ABACUS: a technique for automated behavioral synthesis of approximate computing circuits. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition, pp. 1–6, March 2014
13. Ranjan, A., Raha, A., Venkataramani, S., Roy, K., Raghunathan, A.: ASLAN: synthesis of approximate sequential circuits. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition, pp. 1–6, March 2014
14. Scarabottolo, I., Ansaloni, G., Pozzi, L.: Circuit carving: a methodology for the design of approximate hardware. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition, pp. 545–550, March 2018

15. Schlachter, J., Camus, V., Palem, K.V., Enz, C.: Design and applications of approximate circuits by gate-level pruning. IEEE Trans. Very Large Scale Integr. VLSI Syst. **25**(5), 1694–1702 (2017)
16. Sengupta, D., Snigdha, F.S., Hu, J., Sapatnekar, S.S.: SABER: selection of approximate bits for the design of error tolerant circuits. In: Proceedings of the 54th Design Automation Conference, p. 72, June 2017
17. Vassiliadis, V., et al.: Towards automatic significance analysis for approximate computing. In: Proceedings of the 14th International Symposium on Code Generation and Optimization, pp. 182–193. IEEE, March 2016
18. Venkataramani, S., Roy, K., Raghunathan, A.: Substitute-and-simplify: a unified design paradigm for approximate and quality configurable circuits. In: Proceedings of the Design, Automation and Test in Europe Conference and Exhibition, pp. 1367–1372, March 2013
19. Venkataramani, S., Sabne, A., Kozhikkottu, V., Roy, K., Raghunathan, A.: SALSA: systematic logic synthesis of approximate circuits. In: Proceedings of the 49th Design Automation Conference, pp. 796–801, June 2012